

Maps are an ordered data structure that holds the data in an ordered or sorted form so that elements can easily be looked up in this dictionary-like data structure. A map stores key-value pairs, and these key-value combinations are sorted in increasing order based on their keys.

The keys are like an index operator. In an array, the indexes are integers, while it is not always this case in Maps. A map cannot contain duplicate keys. Each key maps to one value.

Like vectors, maps are dynamic with an automatic resizing feature that automatically grows or shrinks itself when an element is inserted or removed.

1. Map Declaration

To use `map`, we need to include the `<map>` header file in our program.

Here is how we declare a `map`:

```
map<key_type, value_type> map_name;
```

For example, create an empty map:

```
map<string, int> scores;
```

Here, we declared a map named `scores`. The map will have a string as key datatypes and integer as values datatype.

2. Map Initialization

If you want to create a map of integers with initialized values, you can initialize the map in the following two ways, for example:

```
map<string, int> map1 = {{"Math", 90}, {"Physics", 75}, {"Chemistry", 80}};
```

or

```
map<string, int> map1 {{"Math", 90}, {"Physics", 75}, {"Chemistry", 80}};
```

You can copy from another map. For example,

```
map<int, string> map1 = {{1, "first"}, {2, "second"}, {3, "third"}};
map<int, string> map2(map1);
```

or

```
map<int, string> map2 = map1;
```

3. Map Methods

Here are some common map methods:

Method	Description
<code>size()</code>	returns the number of elements present in a map
<code>begin()</code>	returns an iterator that points to the first element of a map.
<code>end()</code>	returns an iterator that points to the past-the-end element of a map. i.e., <code>map.end() - 1</code> is the iterator that points to the last element of the map.
<code>at()</code>	returns reference to the element present at a specified key in a map.
<code>insert()</code>	deletes the item at the position pointed by the iterator.
<code>find()</code>	It search for an element by key and returns the iterator that points to the element if it is found. Otherwise, it will return <code>map.end()</code> .
<code>erase()</code>	deletes the element at the position pointed by the iterator.
<code>empty()</code>	returns <code>true</code> if the map is empty.
<code>clear()</code>	deletes all elements in a map.

4. Basic Map Operations

The map class provides various methods to perform different operations on maps. We will look at some commonly used map operations:

- Access elements
- Iterate through a map
- Add elements
- Update elements
- Delete elements
- ...

4.1 Accessing Map Elements

To access to a map element using keys, simply write an expression:

```
map_name[key];
```

or

```
map_name.at(key);
```

Example 01: Access the elements in a map by key.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> map1 = {"Math", 90}, {"Physics", 75}, {"Chemistry", 80};

    cout << map1["Math"] << " ";
    cout << map1.at("Physics") << " ";

    return 0;
}
```

Output:

90 75

Note that the `at()` method is preferred over `[]` because the `at()` method throws an exception whenever the map is out of bound, while `[]` gives a garbage value.

4.2 Iterating Through a Map

You iterate through map elements by key or via iterators using loops.

The simplest way to iterate through a map is to use the **range-based for loop** (introduced in C++11) along with the `auto` keyword.

Example 02: Iterate through a map using the range-based `for` loop.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, string> map1 = {{3, "Orange"}, {1, "Yellow"}, {5, "Blue"}};

    for(map<int, string>::value_type element: map1)
        cout << element.first << ":" << element.second << "\t";

    return 0;
}
```

Output:

1:Yellow 3:Orange 5:Blue

`first` is the key and `second` is the value.

The output in the above example does not follow the order we initialized because maps are sorted by keys.

In C++, `auto` is often used in loop because the exact type of `map1`'s elements is a bit long and not very intuitive.

Example 03: Iterate through a map using the range-based `for` loop.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, string> map1 = {{3, "Orange"}, {1, "Yellow"}, {5, "Blue"}};

    for(auto element: map1)
        cout << element.first << ":" << element.second << "\t";

    return 0;
}
```

Output:

```
1:Yellow    3:Orange    5:Blue
```

Example 04: Access the elements in a map using a map iterator and `while` loop.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<char, string> map1 = {{'o', "Orange"}, {'y', "Yellow"}, {'b', "Blue"}};

    auto iter = map1.begin(); // create an iterator that points to the first
                             // element of the map
    //map<char, string>::iterator iter = map1.begin();

    //Display the element
    while(iter != map1.end()) {
        cout << iter->second << "\t";

        iter++;
    }
    return 0;
}
```

Output:

```
Blue  Orange  Yellow
```

4.3 Adding Map Elements

To add a single element into a map, we use the `insert()` method or operator `[]`.

The `insert()` method inserts a new element into the map container if the key is not already present.

Example 05: Add elements to a map using the `insert()` method.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}};

    map1.insert({5, 'C'});
    map1.insert({1, 'D'}); // will not be inserted

    for(auto e : map1)
        cout << "{" << e.first << ", " << e.second << "} " << endl;

    return 0;
}
```

Output

```
{1, A}
{2, B}
{5, C}
```

In the above example, the `insert()` method does not insert the specified key-value pair into the map as the key already exists. To overwrite the old value for an existing key, we can use operator `[]`.

Example 06: Add elements to a map using the operator `[]`.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'C'}};

    map1[5] = 'D';
    map1[1] = 'E';

    for(auto e : map1)
        cout << "{" << e.first << ", " << e.second << "} " << endl;

    return 0;
}
```

Output

```
{1, E}  
{2, B}  
{3, C}  
{5, D}
```

4.4 Updating Map Elements

We can access to a map element using square brackets [] or `at()` method and change its value.

Example 07: Update elements in a map.

```
#include <iostream>  
#include <map>  
using namespace std;  
int main() {  
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'C'}};  
  
    map1[1] = 'P';  
    map1.at(2) = 'Q';  
  
    for(auto e : map1)  
        cout << "{" << e.first << ", " << e.second << "} " << endl;  
  
    return 0;  
}
```

Output

```
{1, P}  
{2, Q}  
{3, C}
```

4.5 Deleting Map Elements

To delete a single element from a map, we use the `erase()` method.

Syntax:

```
map_name.erase(key);
```

```
map_name.erase(position);
```

`position` specifies the iterator that is the reference to the position of the element to be erased.

Example 08: Delete elements in a map by key.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'C'}};

    map1.erase(1);
    map1.erase(2);

    for(auto e : map1)
        cout << "{" << e.first << ", " << e.second << "} " << endl;

    return 0;
}
```

Output

```
{3, C}
```

Example 09: Delete elements in a map by key using iterator and `find()` method.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'B'}};

    auto iter = map1.find(2);
    map1.erase(iter);

    for(auto e : map1)
        cout << "{" << e.first << ", " << e.second << "} " << endl;

    return 0;
}
```

Output

```
{1, A}
{3, B}
```

4.6 Sorting a Map

Sorting in a map is not always straightforward. It needs a comparison function object. If the comparison object is ignored, default sorting takes place.

Sorting a map can be done by the following ways:

- Sorted During Creation
- Sorting by value

4.6.1 Sorted During Creation

You can define the order of sorting of map during creation using `less<key_type>` and `greater<key_type>`

Example 10: Sort map based on key in ascending order.

```
#include <iostream>
#include <map>
using namespace std;

int main(){
    map<int, char, less<int>> map1 = {{3, 'B'}, {1, 'A'}, {4, 'D'}, {2, 'C'}};

    for(auto e: map1)
        cout << "{" << e.first << ", " << e.second << "}" << endl;

    return 0;
}
```

Output:

```
{1, A}
{2, C}
{3, B}
{4, D}
```

A map is, by default, created sorted ascending order by keys during creation. Which means, in the above example, you do not need to use the `less<int>` in the program to have it sorted that way.

If you want to change this default sorting order to have it sorted in descending order, you can use `greater<key_type>`

Example 11: Sort map based on key in descending order.

```
#include <iostream>
#include <map>
using namespace std;

int main(){
    map<int, char, greater<int>> map1 = {{3, 'B'}, {1, 'A'}, {4, 'D'}, {2, 'C'}};

    for(auto e: map1)
        cout << "{" << e.first << ", " << e.second << "}" << endl;

    return 0;
}
```

Output

```
{4, D}
{3, B}
{2, C}
{1, A}
```

4.6.2 Sorting a Map by Value

We can achieve this by flipping the key-value pairs (i.e., using keys as values and their corresponding values as keys) and sorting on the basis of keys. But it will only work if the map has distinct values.

To solve this issue, we can use a `multimap`. A multimap in C++ is similar to a map, but it can store elements with duplicate keys.

Example 12: Sort map values in ascending order.

```
#include <iostream>
#include <map>
using namespace std;

int main(){
    map<int, char> map1 = {{3, 'D'}, {1, 'C'}, {4, 'B'}, {2, 'D'}};

    multimap<char, int> multimap1;
    //multimap<char, int, greater<char>> multimap1;

    //Copy elements to the multimap by flipping the key-value pairs
    for(auto e: map1)
        multimap1.insert({e.second, e.first});
    // multimap1[e.second] = e.first // multimap does not support this syntax
}
```

```

    //Display the elements
    for(auto e: multimap1)
        cout << "{" << e.second << ", " << e.first << "}" << endl;

    return 0;
}

```

Output:

```

{4, B}
{1, C}
{2, D}
{3, D}

```

5. Map and Functions

5.1 Passing a Map to a Function

When a map is passed to a function, a copy of the map is created. This new copy of the map is then used in the function and thus, any changes made to the map in the function do not affect the original map.

Example 13: Pass a map to a function (Pass by value)

```

#include <iostream>
#include <map>
using namespace std;
void func(map<int, char> map1){
    map1.insert({4, 'D'});
}
int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'C'}};

    func(map1);

    //Display the elements
    for(auto e: map1){
        cout << e.second << " ";
    }
    return 0;
}

```

Output:

```

A B C

```

Example 14: Pass a map to a function (Pass by reference using the address operator &)

```
#include <iostream>
#include <map>
using namespace std;

void function1(map<int, char> &map1){
    map1.insert({4, 'D'});
}

int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'C'}};

    function1 (map1);

    //Display the elements
    for(auto e: map1)
        cout << e.second << " ";

    return 0;
}
```

Output

A B C D

5.2 Returning a Map from a Function

Example 15: Return a map from a function

```
#include <iostream>
#include <map>
using namespace std;

map<int, char> function1(map<int, char> map1){
    map1.insert({4, 'D'});

    return map1;
}

int main() {
    map<int, char> map1 = {{1, 'A'}, {2, 'B'}, {3, 'C'}};

    map1 = function1(map1);

    // Display the elements
    for(auto e: map1)
        cout << e.second << " ";

    return 0;
}
```

Output

A B C D

6. Map of Vectors

Maps can store vectors and vice versa.

Example 16: In this example, we have a map whose values are vectors.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main(){
    map<int, vector<string>> map1 = {
        {1, {"Black", "Blue", "Yellow"}},
        {2, {"Red", "Orange", "Green"}},
        {3, {"Purple", "Pink", "White"}}
    };

    // Display the elements
    for(auto e : map1) {
        cout << e.first << " ";

        for(string i : e.second)
            cout << i << " ";

        cout << endl;
    }
    return 0;
}
```

Output:

```
1 Black Blue Yellow
2 Red Orange Green
3 Purple Pink White
```

Example 17: In this example, we have a map whose keys are vectors.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main(){
    map<vector<string>, int> map1 = {
        {"Black", "Blue", "Yellow"}, 1},
        {"Red", "Orange", "Green"}, 2},
        {"Purple", "Pink", "White"}, 3}
    };

    //Display the elements
    for(auto e : map1) {
        for(string color : e.first)
            cout << color << " ";

        cout << e.second << endl;
    }
    return 0;
}
```

Output:

```
Black Blue Yellow 1
Purple Pink White 3
Red Orange Green 2
```

7. Two-dimensional Maps

A map can be placed inside another map to create a multi-dimensional map. Multidimensional maps refer to maps of maps.

A 2D map is a map of the map. Like 2D arrays, we can declare and assign values to a 2D map. By placing a map specifier within the `type` parameter of another map, you may specify a Two-dimensional map.

7.1 Declaring a 2D Map

```
map<key_type, map<key_type, value_type>> map_name;
```

For example:

```
map<string, map<string, int>> map2D;
```

Here, we create a two-dimensional map. The keys in this map are strings and the values are maps. These inner maps have keys of type `string` and `int` values.

7.2 Initializing a 2D Map

A table below shows how a 2D Map can store data. 2D Maps are very useful when there is nested information. For example, in the table below, each person has an identifier "name" and has several pieces of nested information (age, height and weight).

Name	Information
John	Age: 50 Height: 170 Weight: 70
Mary	Age: 35 Height: 150 Weight: 50

```
map <string, map<string, int>> map2D = {  
    {"John", {{ "Age", 50}, {"Height", 170}, {"Weight", 70}}},  
    {"Mary", {{ "Age", 35}, {"Height", 150}, {"Weight", 50}}},  
};
```

The number of elements in the inner maps of a map does not necessarily need to be the same. For example:

```
map <string, map<string, int>> map2D = {  
    {"John", {{ "Age", 50}, {"Height", 170}, {"Weight", 70}}},  
    {"Mary", {{ "Age", 35}, {"Height", 150}}},  
};
```

7.3 Accessing Two-dimensional Maps

Example 18: Access 2D map elements.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, map<string, int>> map2D = {
        {"John", {{ "Age", 50}, {"Height", 170}, {"Weight", 70}}},
        {"Mary", {{ "Age", 35}, {"Height", 150}, {"Weight", 50}}},
    };

    cout << map2D["John"]["Age"] << endl;
    cout << map2D.at("Mary").at("Height") << " ";

    return 0;
}
```

Output

```
50
150
```

7.4 Iterating Through a 2D Map

Example 19: Iterate through a 2D map using the range-based **for** loop.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, map<string, int>> map2D = {
        {"John", {{ "Age", 50}, {"Height", 170}, {"Weight", 70}}},
        {"Mary", {{ "Age", 35}, {"Height", 150}, {"Weight", 50}}},
    };

    // Display the elements
    for(auto person: map2D){
        cout << person.first << " ";
        for(auto attribute : person.second)
            cout << attribute.first << ": " << attribute.second << "\t";

        cout << endl;
    }
    return 0;
}
```

Output

```
John Age: 50  Height: 170  Weight: 70
Mary Age: 35  Height: 150  Weight: 50
```

7.5 Adding Two-dimensional Map Elements

Example 20: Add elements to a 2D map using the `insert()` method and operator `[]`.

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, map<char, int>>> map2D = {
        {1, {{ 'A', 20}, { 'B', 25}, { 'C', 30}}},
        {2, {{ 'A', 10}, { 'B', 15}, { 'C', 20}}}
    };

    map2D.insert({3, {{ 'A', 5}, { 'B', 10}}});
    map2D[4] = {{ 'A', 50}, { 'B', 30}, { 'C', 20}, { 'D', 25}};
    map2D[2]['D'] = 50;

    //Display the elements
    for(auto person: map2D){
        cout << person.first << " ";
        for(auto attribute : person.second){
            cout << attribute.first << ": " << attribute.second << "\t";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
1 A: 20 B: 25  C: 30
2 A: 10 B: 15  C: 20  D: 50
3 A: 5  B: 10
4 A: 50 B: 30  C: 20  D: 25
```


Example 21: Store elements in a 2D map by user input.

```
#include <iostream>
#include <map>
using namespace std;

int main(){
    map <int, map<char, int>> map2D;

    int size;
    char key;
    int value;

    cout<< "Enter the size for a square matrix: ";
    cin >> size;

    //Enter the elements
    for (int i = 0; i < size; i++){
        for (int j = 0; j < size; j++){
            cout << "Enter key and value of map1[" << i << "][" << j << "]: ";
            cin >> key;
            cin >> value;

            map2D[i].insert({key, value});
        }
    }

    // Display the elements
    for(auto m : map2D) {
        for(auto e : m.second)
            cout << e.second << " ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
Enter the size for a square matrix: 2
Enter key and value of map1[0][0]: 1 10
Enter key and value of map1[0][1]: 2 20
Enter key and value of map1[1][0]: 3 30
Enter key and value of map1[1][1]: 4 40
10 20
30 40
```

7.6 Deleting Two-dimensional Map Elements

Example 22: Delete elements from a 2D map using `erase()` method.

```
#include <iostream>
#include <map>
using namespace std;

int main(){
    map<int, map<char, int>> map2D = {
        {1, {{ 'A', 10}, { 'B', 11}, { 'C', 12}}},
        {2, {{ 'A', 13}, { 'B', 14}, { 'C', 15}}},
        {3, {{ 'A', 16}, { 'B', 17}, { 'C', 18}}}
    };

    // Delete the elements
    map2D[1].erase('A');
    map2D.erase(2);

    // Display the elements
    for(auto m : map2D) {
        for(auto e : m.second)
            cout << e.second << " ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
11 12
16 17 18
```

Exercises

Write the programs below using map; you are NOT allowed to use arrays, vectors, structures nor user-defined classes.

1. (*Guess the capitals*) Re-write the Guess capitals program. Ten countries and their capital cities are stored in a map. Write a program that asks the user to answer all the capital cities and displays the total correct count. The user's answer is case-insensitive. Here is a sample run:

```
What is the capital city of Cambodia? Phnom Pehn Enter
You answer is correct
What is the capital city of Thailand? Beiging Enter
The correct answer should be Bangkok
What is the capital city of China? ...
...
The correct count is 8.
```

2. Write a function called `get_class_average_age` that has one parameter called `students` which is a map containing three students with ids, names and ages. This function will return the average age of the students in the class.
3. Write a function called `count_char` that will accept a sentence and return a `map<char, int>` that stores the frequency of each character. The counting must be case-insensitive. Create another function called `display_char_count` that will accept the `map<char, int>` and display the result. Here is a sample run:

```
Please enter a sentence: Apple and Banana Enter
A: 5
B: 1
C: 0
...
Z: 0
```

4. Ask the user to enter an integer number in between 0 and 999, and displays the it in English. Here is a sample run:

```
Enter a number: 1500 Enter
Please enter a number between 0 and 999. Try again!
Enter a number: 456 Enter
Your number is: four hundred fifty-six
```

5. Write a program that stores ten students (id starts from 1, name, and age between 18 to 22) in a map. Display a menu that will allow the user to select any of the following menus:
 - a. Sort students by name.
 - b. Sort students by age
 - c. Exit the program

When display the student(s), the data should be arranged in a tabular format, for example:

ID	Name	Age
5	Chan Dara	20
2	Keo Tola	20
4	Sok Sophea	20

6. Write a simple ATM program. Store five account store them in a map. When the program starts, ask the user to enter the account number and password login. If login succeeded, display the following menus:
 - a. Balance
 - b. Withdraw
 - c. Deposit
 - d. Exit the program

Note:

- If the login failed, repeat it again and again; do not end the program there.
- If the balance is not enough for withdrawing, ask the user if they want to enter another amount.

7. Write a program to store id, name, salary and department of three employees in a 2D map. Also include a menu that will allow user to select any of the following features:
 - a. Display all employees
 - b. Add a new employee
 - c. Delete employee by id
 - d. Update employee by id
 - e. Search employee by id
 - f. Exit the program

When display the employee(s), the data should be arranged in a tabular format, for example:

ID	Name	Salary	Department
1	Chan Dara	1000	ITE
2	Sok Sophea	1200	DSE
3	Keo Tola	800	TEE

8. (*Financial tsunami*) Banks loan money to each other. If a bank goes bankrupt, it would not be able to pay back the loan. A bank's total asset is its current balance plus its loans to other banks. Figure 8.7 is a diagram that shows five banks. The current balance of the bank 0 is 25, and its total asset is: $25 + 100.5 + 320.5 = 446$ million dollars. The directed edge from node 0 to node 1 indicates that bank 0 loans 100.5 million dollars to bank 1.

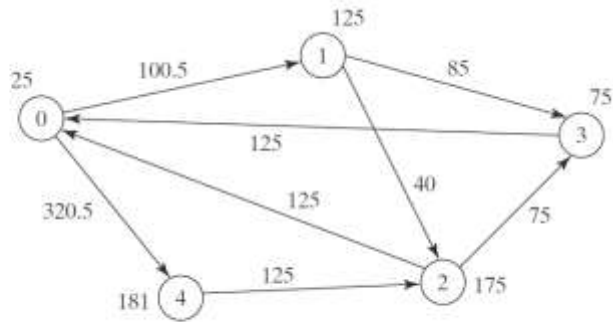


FIGURE 8.7 Banks loan money to each other.

If a bank's total asset is under a certain limit, the bank is unsafe. **If a bank is unsafe, the money it borrowed cannot be returned to the lender, and the lender cannot count the loan in its total asset.** Consequently, the lender may also be unsafe if its total asset is under the limit. Write a program to display all unsafe banks above given that the limit is 201 million dollars.

Hint: Use a map to represent the loans as below:

```
map<int, map<int, int>> loan = {
    {0, {{1, 100.5}, {4, 320.5}}},
    {1, {{2, 40}, {3, 85}}},
    {2, {{0, 125}, {3, 75}}},
    {3, {{0, 125}}},
    {4, {{2, 125}}}
```

```
};
```

`loan[i][j]` indicates the loan or money that bank `i` lends to bank `j`. Once bank `j` becomes unsafe, `loan[i][j]` should be set to 0 or deleted.

Reference

- [1] Y. Daniel Liang. 'Introduction to Programming with C++', 3e – 2014
- [2] Dey, P., & Ghosh, M. 'Computer Fundamentals and Programming in C', 2e – 2013
- [3] <https://cplusplus.com/doc/tutorial/>
- [4] <https://www.programiz.com/cpp-programming>
- [5] <https://www.studytonight.com/cpp/>